Characterizing the Feasible Region for Self-Modifying Substrates in Interactive Environments

Avir Research

2026-03-19

The shared artifact. Birth writes theory. Experiment writes results. Compress edits both.

Abstract

We formalize six rules (R1-R6) for recursive self-improvement as mathematical conditions on a state-update function $f : S \times X \to S$ and derive necessary properties of any system satisfying all six simultaneously. From 546+ experiments across 11 architecture families on ARC-AGI-3 interactive games, we extract 26 constraints and prove: (1) no satisfying system has a fixed point — self-modification is necessary, not optional; (2) in finite environments, the system must process its own internal state to maintain irredundant growth (the self-observation requirement); (3) the feasible region is non-empty for Level 1 navigation but currently unoccupied for the full constraint set including R3 (self-modification of operations). Whether a substrate exists inside all six walls remains open. The contribution is the walls themselves.

1. Introduction

546+ experiments across 11 architecture families (codebook/LVQ, LSH, L2 k-means, reservoir, graph, connected-component, Bloom filter, kd-tree, cellular automata, LLM, Recode/self-refining LSH) tested substrates for navigation and classification on ARC-AGI-3 interactive games and a cross-domain chain benchmark (CIFAR-100 → ARC-AGI-3 → CIFAR-100). All experiments used the same evaluation framework (R1-R6) and constraint map.

The experiments carved a feasible region — the set of systems that satisfy all constraints simultaneously. This paper formalizes the constraints mathematically, derives necessary properties of the feasible region, and states honestly what is proven vs conjectured vs open.

The paper is valid regardless of outcome. If a substrate satisfying all six rules is found, the feasible region is non-empty and the characterization guided the search. If no such substrate exists, the characterization identifies which constraints are mutually exclusive — also a contribution. We report whichever is true at time of writing.

2. Related Work

2.1 Self-referential self-improvement

Schmidhuber's Gödel Machine (2003) is the closest formal framework: a self-referential system that rewrites its own code when it can prove the rewrite is useful. Key differences from our framework: (1) Gödel machines require a utility function (external objective — our R1 prohibits this), (2) provable self-improvement is limited by Gödel's incompleteness theorem, (3) the Gödel machine framework doesn't address exploration saturation in finite environments.

2.2 Intrinsic motivation and curiosity-driven exploration

Pathak et al. (2017) formalize curiosity as prediction error in a learned feature space. Count-based exploration (Bellemare et al., 2016) uses state visitation counts. Both address exploration in sparse-reward environments. Key difference: these methods add intrinsic REWARD signals, which function as objectives. Our R1 requires no objectives. Our derivation shows self-observation is required by the constraint system itself, not as a reward mechanism.

2.3 Autopoiesis

Maturana & Varela (1972) define autopoietic systems as networks that produce the components that produce the network. Organizationally closed. Related to our fixed-point conjecture (Section 4.4). Key difference: autopoiesis maintains structure (homeostasis); our U17 requires unbounded growth. Our system is autopoietic + growth.

2.4 Growing neural gas and self-organizing maps

Fritzke (1995) GNG, Kohonen (1988) SOM/LVQ. The codebook substrate is LVQ + growing codebook. Well-characterized in the literature. Our contribution is not the architecture — it's the constraint map extracted from systematic testing.

2.5 Graph-based exploration for ARC-AGI-3

Rudakov et al. (2025) independently developed a training-free graph-based exploration method for ARC-AGI-3, ranking 3rd on the private leaderboard (30/52 levels across 6 games). Their method uses connected-component segmentation for frame encoding, hierarchical action prioritization (5 tiers by visual salience), and BFS path planning to frontier states. Key parallels: (1) graph-structured state tracking with hash-based node identity — equivalent to our $\pi : X \to N$; (2) click-space as expanded action space (4,096 actions for click games) — same finding as our Steps 503/505; (3) frontier-directed exploration — this IS the purposeful exploration (I6/I9) we identified as the Level 2 bottleneck, and they implemented it successfully. Key differences: their method is training-free (no learning, no self-modification) and assumes deterministic, fully observable environments. It satisfies R1 but not R3 — the graph accumulates but the exploration strategy is fixed. Their limitation ("exhaustive exploration becomes computationally intractable" at higher levels) aligns with our Theorem 2: without self-observation, exploration saturates.

2.6 Object-centric Bayesian game learning

Heins et al. (2025, "AXIOM") combine object-centric scene decomposition with online Bayesian model expansion and reduction, learning games in ~10K steps without backpropagation. Key mechanisms: (1) Slot Mixture Model segments frames into objects with position/color/shape — far richer than spatial averaging; (2) online expansion grows mixture components when new data doesn't fit (equivalent to spawn-on-novelty); (3) Bayesian Model Reduction (BMR) merges redundant components every 500 frames — consolidation that prevents centroid explosion while preserving information. AXIOM outperforms DreamerV3 and BBF on 10/10 games at 10K steps. Key differences from our framework: AXIOM uses reward signals for planning (violates R1), operations are fixed (violates R3), and BMR requires a prescribed merge criterion (frozen frame). However, BMR addresses our U3-vs-R6 tension directly: U3 says never delete, R6 says no redundancy, BMR resolves this by merging (not deleting) redundant structure.

2.7 Continual learning and catastrophic forgetting

McCloskey & Cohen (1989) identified catastrophic forgetting — neural networks lose previous knowledge when learning new tasks. The continual learning literature (survey: van de Ven & Tolias, 2024, arXiv:2403.05175) identifies six main approaches: replay, parameter regularization, functional regularization, optimization-based, context-dependent processing, and template-based classification. All assume a neural network with backpropagation (violates R1/R2). Our chain benchmark (Section 5.4) tests forgetting WITHOUT any mitigation mechanism — the substrate must naturally resist forgetting through its dynamics alone (U3: zero forgetting by construction, not by regularization).

3. Formal Framework

3.1 Primitives

The substrate is a triple (f, g, F): - f_s: X → S (state update parameterized by current state s) - g: S → A (action selection) - F: S → (X → S) (meta-rule mapping states to update rules; F IS the frozen frame)

Dynamics: s_{t+1} = F(s_t)(x_t), a_t = g(s_t).

3.2 Structural Rules (R1-R3) and the Core Tension

Prior work: R1 (no external objectives) is the standard unsupervised/self-supervised setting. R2 (adaptation from computation) rules out external optimizers — related to Hebbian learning (Hebb, 1949) where adaptation is local and intrinsic. R3 (self-modification) is formalized by Schmidhuber (2003) as the Gödel machine's self-referential property, though his version requires a proof searcher we do not.

Our formalization:

- **R1:** $f_s(x)$ depends only on s and x. No external signal L enters. The substrate is a closed dynamical system over S given input stream X.

- **R2:** All parameters $\Theta(f) \subseteq S$. The only mechanism modifying Θ is f itself. No gradient ∇L, no external optimizer. The map $t \mapsto s_t$ is generated entirely by iterating f.
- **R3:** $F : S \to (X \to S)$ is non-constant. $\exists s_1 \neq s_2$ such that $F(s_1) \neq F(s_2)$ as functions on X. The update rule depends on the state, not just the data.

Core Tension (Theorem 1): R3 + U7 (dominant amplification) + U22 (convergence kills exploration) produce convergence pressure. U17 (unbounded growth) prevents convergence. Proof: if $s_t \to s^*$, then $\phi(s^*) < \infty$, contradicting U17. Therefore the system has no fixed point. Self-modification is necessary — growth perpetually disrupts the dominant mode.

Relationship to prior work: The no-fixed-point result is a consequence of combining standard dynamical systems theory (spectral convergence) with the growth axiom (U17). The individual pieces are known; the combination producing perpetual non-convergence appears to be our derivation. Closest prior work: Schmidhuber's "asymptotically optimal" self-improvement, which converges — our system provably doesn't.

Tensions: - T1: U7 assumes stationarity that R3 breaks. May need reformulation as "locally amplifies largest-variance component" (instantaneous, not asymptotic). - T2: R3 is ambiguous between "actively modifies operations" and "operations responsive to state." Resolved by the self-modification hierarchy below.

Resolving T2: The Self-Modification Hierarchy **Prior work:** Schmidhuber (1993, "A Self-Referential Weight Matrix") proposed collapsing potentially infinite meta-learning levels into one self-referential system. Kirsch & Schmidhuber (2022, ICML) implement this: a weight matrix that learns to modify its own weights, achieving meta-meta-. . . -learning without explicit hierarchy. The hierarchy of meta-learning levels is well-established: Level 0 (fixed), Level 1 (learning), Level 2 (learning to learn), etc. Our contribution is mapping this hierarchy onto our specific decomposition of F.

Our formalization: Decompose $F(s)(x)$ into three components: - $\pi_s : X \to N$ (encoding — maps observations to nodes) - $u_s : N \times S \to S$ (update — modifies state given a node) - $g_s : S \to A$ (selection — chooses actions)

The **self-modification level** of a substrate is determined by which components' *structure* (not just data) depends on s:

Level	What depends on s	Example	R3?
ℓ_0	Only g_s (action from data)	LSH: fixed hash, argmin over edge counts	Weak only
ℓ_1	g_s and data in u_s	Codebook: entries move via attract, but attract rule fixed	Weak only

Level	What depends on s	Example	R3?
ℓ_π	π_s changes structure	Recode: hash routes through learned hyperplanes from transition statistics	**Partial**
ℓ_F	The rule for modifying π itself adapts	No substrate yet. Schmidhuber's SRWM is Level ℓ_F for neural systems.	**Strong**

Key distinction: At ℓ_0 and ℓ_1, the encoding π is fixed at initialization. The system sees through the same lens forever; only what it remembers changes. At ℓ_π, the lens itself changes — an observation that mapped to node n_1 at time t may map to $(n_1, 0)$ at time $t' > t$ because a hyperplane was learned. This is a qualitative jump: the system's perception of the environment changes, not just its memory.

Empirical evidence (Step 542, Recode): ℓ_π produces 5/5 L1 navigation vs 3/3 for ℓ_0 (LSH k=16). Self-modification of π improves navigation reliability. But the refinement algorithm (entropy threshold, hyperplane from mean difference) is frozen — the system cannot modify HOW it refines, only WHERE. This is partial R3: F is non-constant because π_s varies with s, but F is still a SINGLE fixed meta-rule.

Relationship to prior work: Schmidhuber's hierarchy (1993) addresses the same question — what level of self-reference does the system have? His solution (self-referential weight matrix) collapses all levels into one. Our framework makes the levels explicit and maps concrete substrates to them. The contribution is empirical: showing that ℓ_π is achievable and produces measurable improvement over ℓ_0, while ℓ_F remains open.

Implication for Theorem 2: Self-observation (Section 4.3) requires extracting new irredundant structure from s. At ℓ_0, the system cannot encode this new structure into π — it can only add edges. At ℓ_π, it can modify π to distinguish states that were previously confused. ℓ_π is necessary but may not be sufficient for full self-observation — the system also needs to modify its update and selection rules (ℓ_F).

3.3 Growth Topology (U3, U17, U20, R6)

Prior work: U3 (zero forgetting) is the catastrophic forgetting constraint from continual learning (McCloskey & Cohen, 1989; French, 1999). U20 (local continuity) is Lipschitz continuity of the mapping — standard in topology-preserving embeddings (Kohonen, 1988; van der Maaten & Hinton, 2008 for t-SNE). R6 (no deletable parts) relates to minimal sufficient statistics (Fisher, 1922).

Our formalization:

- **U3:** $S_t \subseteq S_{t+1}$ (structural inclusion — components added, never removed). Values may change; skeleton only grows.
- **U17:** $\exists \phi : S \to \mathbb{R}_{\geq 0}$ monotonically non-decreasing, with $\lim_{t\to\infty} \phi(s_t) = \infty$.

- **U20:** $\pi : X \to N$ is Lipschitz: $d_N(\pi(x_1), \pi(x_2)) \leq L \cdot d_X(x_1, x_2)$.
- **R6:** For every component $c \in \text{components}(S_t)$, the restricted state $S_t \setminus \{c\}$ fails the ground truth test G.

Propositions: 1. U3 + U17 + R6 $\Rightarrow$ irredundant growth (every new component covers unique territory). 2. U20 + irredundancy $\Rightarrow$ well-separated nodes (Voronoi cells partition X without unnecessary overlap). 3. Growth rate is coupled to observation distribution (environment gates growth). 4. U20 constrains node topology to inherit from X (connected observations $\to$ connected nodes).

Relationship to prior work: The individual constraints are known (catastrophic forgetting, Lipschitz continuity, minimal sufficiency). The combination producing "irredundant growth" — where every new component is both unique and necessary — appears to be our synthesis.

Tension T3: R3 (self-modifying metric) + U20 (continuous mapping) = metric can REFINE (increase resolution) but cannot REARRANGE (swap what's near/far). This constrains the space of allowed self-modifications.

Tension T4: Finite observation space + irredundant growth = node count is bounded. U17 must be satisfied by something other than nodes (edges, in practice). But edge-count growth after node saturation violates R6 (marginal counts are redundant). This leads to Theorem 2 (Section 4.3).

3.4 Action Selection Constraints (U1, U11, U24)

U1: No separate learning and inference modes **Prior work:** Online continual learning (OCL) formalizes exactly this constraint. Aljundi et al. (CVPR 2019, "Task-Free Continual Learning") define systems that learn from a single-pass data stream with no task boundaries. The broader OCL literature (survey: arXiv:2501.04897) emphasizes "real-time adaptation under stringent constraints on data usage." Our U1 is equivalent to the OCL setting.

Our formalization: F is time-invariant. The same meta-rule $F : S \to (X \to S)$ applies at every timestep. There is no mode variable $m \in \{train, infer\}$ that changes F's behavior. Formally: the system's dynamics are a single autonomous dynamical system, not a switched system.

Relationship to prior work: Equivalent to OCL's single-pass constraint. Not novel — properly attributed.

Implications: Combined with R3 (self-modification), U1 says: the system modifies itself using the SAME function it uses to process input. There is no separate training phase where the system is allowed to self-modify more aggressively. This rules out any architecture with explicit "learning rate schedules" or "warmup phases" unless these are derived from the state itself (which would make them R3-compliant).

U24: Exploration and exploitation are opposite operations **Prior work:** The exploration-exploitation tradeoff is foundational in RL (Sutton & Barto, 2018). The formal impossibility of a single mechanism optimizing both simultaneously is well-established in multi-armed bandit theory (Auer et al., 2002; Lai & Robbins, 1985). Recent work (arXiv:2508.01287, 2025) suggests exploration can emerge from pure exploitation under specific conditions (repeating tasks, long horizons).

Our formalization: Let $g_{explore} : S \to A$ maximize coverage (minimize revisitation) and $g_{exploit} : S \to A$ maximize classification accuracy. Then $g_{explore} \neq g_{exploit}$ in general.

Specifically: $g_{explore} = \text{argmin}_a \sum_n E(c, a, n)$ (least-tried action) and $g_{exploit} = \text{argmax}_a \text{score}(s, a)$ (highest-confidence action). These select opposite actions when the least-explored action is also the least-confident.

Relationship to prior work: This is the standard RL tradeoff. Not novel. Our contribution is empirical confirmation across 546+ experiments that no single g produces both good navigation and good classification (Steps 418, 432, 444b).

U11: Discrimination and navigation require incompatible action selection **Prior work:** No direct formal precedent found. The RL literature treats navigation and classification as different TASKS but doesn't formalize them as requiring incompatible mechanisms within the same system. The closest work is multi-objective RL, where Pareto-optimal policies for conflicting objectives are studied (Roijers et al., 2013).

Our formalization: Navigation requires $g_{nav}(s) = \text{argmin}_a \text{count}(s, a)$ — the action that maximizes coverage. Classification requires $g_{class}(s) = \text{argmax}_a \text{score}(s, a, \text{label})$ — the action (label assignment) that maximizes match confidence. These are not just "different" — they are NEGATIONS of each other (argmin vs argmax over the same scoring function applied to the same state).

Relationship to prior work: The specific finding — that argmin produces 0% classification (Step 418g) and argmax produces 0% navigation — appears to be our empirical contribution, not previously formalized as a constraint. However, the underlying principle (coverage-maximizing and accuracy-maximizing objectives conflict) is a special case of multi-objective optimization theory.

Implications: A system satisfying R1-R6 must handle BOTH tasks (navigation and classification). Since they require opposite g, the system needs either: (a) a mechanism to SWITCH between g_{nav} and g_{class} based on context — but U1 forbids mode switching; or (b) a single g that somehow serves both objectives simultaneously — but U24 says this is impossible. This creates a genuine tension.

Degree of freedom 11: How does the system resolve the U1 + U11 + U24 tension? One possibility: g depends on s (which it must, by R3), and the STATE determines whether the system's behavior is more exploratory or more exploitative. This is not mode switching (the function g is the same) — it's state-dependent behavior. The system explores when its state indicates uncertainty, and exploits when its state indicates confidence. This is known in the RL literature as Bayesian exploration (Ghavamzadeh et al., 2015).

3.5 Remaining Structural Rules (R4-R6) and Validated Constraints (U7, U16, U22)

R4: Modification tested against prior state **Prior work:** Regression testing in software engineering (Rothermel & Harrold, 1996). In RL, policy improvement is tested against the previous policy (Kakade & Langford, 2002, conservative policy iteration). In self-adaptive systems, runtime testing validates adaptations against pre-adaptation behavior (Fredericks et al., 2018). The formal requirement that a self-modifying system must self-evaluate is implicit in Schmidhuber's Gödel machine (the proof must show the rewrite improves expected utility).

Our formalization: After each modification $s_{t+1} = f_{s_t}(x_t)$, there exists an evaluation $V : S \times S \times X^* \to \{better, worse, same\}$ applied by f itself (not externally) to novel inputs X^*. V is part of F (frozen).

Relationship to prior work: Equivalent to conservative policy iteration's requirement. The novelty, if any, is requiring V to be part of f's dynamics (R2 compliance) rather than an external evaluator. This is the same distinction we make for R1.

R5: One fixed ground truth **Prior work:** In formal verification, the specification is the fixed point against which the system is tested. In Schmidhuber's Gödel machine, the utility function is the fixed external criterion. In evolution, fitness is determined by the environment (fixed, external). The idea that a self-modifying system needs at least one invariant is well-established — without it, the system can trivially "improve" by redefining improvement.

Our formalization: $\exists!\ G : S \to \{0, 1\}$ (ground truth test) such that $G \notin S$ — the system cannot modify G. G is part of F.

Relationship to prior work: Standard. Not novel.

R6: No deletable parts (minimality) **Prior work:** Minimal realizations in control theory (Kalman, 1963) — a state-space representation with no redundant states. Minimal sufficient statistics (Fisher, 1922). Minimal dynamical systems in topological dynamics — a system where every orbit is dense (Scholarpedia). Our R6 is closest to Kalman's minimal realization: no state can be removed without losing input-output behavior.

Our formalization: For every component $c \in \text{components}(S_t)$: $G(S_t \setminus \{c\}) = 0$. Every element is load-bearing.

Relationship to prior work: Equivalent to Kalman's observability + controllability condition in linear systems. For nonlinear growing systems, we are not aware of a standard formalization. The combination with U17 (unbounded growth) is non-standard — Kalman minimality assumes fixed dimension.

U7: Iterated application amplifies dominant features **Prior work:** Power iteration (von Mises, 1929). In linear systems, repeated application of a matrix amplifies the dominant eigenvector. In neural networks, repeated weight application leads to mode collapse (vanishing/exploding gradients, Bengio et al., 1994). In recurrent networks, the issue is well-studied as the echo state property (Jaeger, 2001).

Our formalization: For linearization Df of f around any trajectory point: $\|f^n(s, \cdot) - \lambda_1^n v_1 v_1^T f(s, \cdot)\| \to 0$ as $n \to \infty$, where λ_1, v_1 are the dominant eigenvalue/eigenvector.

Relationship to prior work: This IS power iteration. Known since 1929. Our contribution is observing it empirically across architectures (codebook Step 405, reservoir Steps 438-439) and noting its interaction with R3 and U17 (Theorem 1).

U16: Input must encode differences from expectation **Prior work:** Mean subtraction / centering is standard preprocessing in machine learning (LeCun et al., 1998, "Efficient BackProp"). In neuroscience, predictive coding (Rao & Ballard, 1999) formalizes the idea that neural systems encode prediction errors, not raw stimuli. Whitening (decorrelation + unit variance) extends centering.

Our formalization: The observation x is preprocessed as $x - \mathbb{E}[x]$ before f_s processes it. In our framework, centering is part of F — it's frozen, load-bearing, and confirmed across 2 families (codebook Step 414, LSH Step 453).

Relationship to prior work: Centering is standard (LeCun 1998). The specific finding that it's load-bearing for TWO architecturally distinct families (codebook: prevents cosine saturation; LSH: prevents hash concentration) is our empirical contribution, not the formalization.

U22: Convergence kills exploration **Prior work:** In bandit theory, a converged policy exploits one arm and never explores others (Lai & Robbins, 1985). In RL, policy convergence to a deterministic policy eliminates exploration (Sutton & Barto, 2018). The formal statement that convergent dynamics in interactive environments produce stationary behavior is well-established.

Our formalization: If $\|s_{t+1} - s_t\| \to 0$, then (a_t) becomes eventually periodic. Mathematically: convergent adaptation makes the environment appear stationary $\to$ trivially predictable $\to$ no learning signal $\to$ no new states.

Relationship to prior work: Known. The specific empirical confirmations (codebook Step 428 score convergence, TemporalPrediction Step 437d weight freezing) are data points for a known phenomenon.

4. Results

4.1 The Core Tension (R3 + U7 + U17 + U22)

Theorem 1: No satisfying system has a fixed point. If $s_t \to s^*$, then $\phi(s^*) < \infty$, contradicting U17. Self-modification is necessary, not optional — growth perpetually disrupts the dominant mode (U7), preventing convergence (U22).

The system oscillates: U7 drives toward dominant mode $\to$ U17 growth disrupts the mode via R3 $\to$ new dominant mode emerges $\to$ repeat. The oscillation period is a degree of freedom.

4.2 Irredundant Growth

U3 + U17 + R6 together require that the system adds new components indefinitely (U17), never removes them (U3), and every component is necessary (R6). Therefore every new component covers unique territory. Combined with U20 (Lipschitz mapping), this means nodes are well-separated — the system builds an increasingly detailed, non-redundant map of the observation space. Growth rate is coupled to the observation distribution: the environment gates growth through the action-observation loop.

4.3 The Self-Observation Requirement

Theorem 2: In a finite environment, U17 + R6 + R1 require self-observation.

Proof: (1) U17 + R6 require infinitely many irredundant components. (2) In a finite environment, irredundant components from external observations are bounded — once every reachable state has a node, further nodes are redundant; edge-count growth is also redundant ($N(c, a, n) = 10^6$ vs

10^6+1 does not change g). R6 is violated. (3) After external information is exhausted, irredundant growth must come from elsewhere. (4) By R1, $f_s(x)$ depends only on s and x. (5) Since x provides no new irredundant information, the only source is s itself. f must extract structure from s not explicitly stored — temporal patterns, graph properties, meta-state. This is self-observation. □

Implication: The current graph + argmin system exits the feasible region after exploration saturates. The Level 2 failure (Section 5.2) is a feasibility violation, not a strategy failure.

Relationship to prior work: Curiosity-driven exploration (Pathak et al., 2017) proposes self-observation as a design choice. We derive it as a mathematical necessity. The R6 mechanism — irredundancy killing degenerate growth — has no analog in the curiosity literature, which adds intrinsic reward (violating R1) rather than requiring every component to be functionally necessary.

4.4 Fixed-Point Conjecture

Conjecture (not proven): The minimal self-observing substrate is a fixed point of F. Applying the system to its own state trajectory reproduces the system's dynamics. This would terminate the potential infinite regress of self-observation (processing state → new state → processing new state → ...). Related to autopoiesis (Maturana & Varela, 1972): the network produces the components that produce the network. Difference: autopoiesis maintains structure; our system grows (U17). Status: open.

4.5 Argmin Robustness and the Noisy TV Barrier

Prior work: The noisy TV problem (Burda et al., 2019, ICLR) identifies a failure mode of curiosity-driven exploration: agents rewarded for prediction error or novelty are attracted to irreducibly stochastic transitions (a "noisy TV") because these transitions can never be predicted accurately. Random Network Distillation (RND, Burda et al., 2018) addresses this by using a fixed random network as a prediction target, making the bonus deterministic. Count-based exploration (Bellemare et al., 2016) is known to be robust to noisy TV because visit counts don't distinguish high-variance from low-variance transitions.

Our finding (empirical, not a theorem): In R1-compliant systems (no external reward signal), the noisy TV problem is universal across ALL targeted exploration strategies, not just curiosity. We tested 6 independent strategies against pure argmin on LS20:

Strategy	Mechanism	Result	Failure mode
Argmin (baseline)	$g(s) = \text{argmin}_a N(s,a)$	10/10	—
Destination novelty	$g(s) = \text{argmax}_a \text{novelty}(s'\|s,a)$	1/10	Attracted to rare death states
Prediction error	$g(s) = \text{argmax}_a \|s' - \hat{s}'\|$	0/10	Death is maximally unpredictable
Softmax temperature	$P(a) \propto \exp(-N(s,a)/T)$	2/3	Stochasticity without benefit
Entropy-seeking	$g(s) = \text{argmax}_a H(s'\|s,a)$	0/3	Noisy TV: death = max entropy
UCB1	$g(s) = \text{argmax}_a(\bar{r}_a + c\sqrt{\ln t/N_a})$	2/3	Degenerates to argmin (no reward)

Strategy	Mechanism	Result	Failure mode
Global novelty	$g(s) = \text{argmax}_a(1/N_{\text{global}}(s'))$	6/10	Same count, different seeds

Steps 477-482, 539-541. 6 strategies tested, all worse than or equal to argmin.

Why argmin is robust: Define a transition as *reducible* if $H(s'|s,a) \to 0$ with sufficient observations, and *irreducible* if $H(s'|s,a) > \epsilon$ for all sample sizes (e.g., death transitions, stochastic resets). Any strategy g that selects actions based on model uncertainty (prediction error, entropy, novelty) preferentially selects actions leading to irreducible transitions, because these maximize the quality signal. Argmin $g(s) = \text{argmin}_a N(s,a)$ is immune: visit counts accumulate equally on reducible and irreducible transitions. The cost is slower exploration of structured regions; the benefit is avoiding traps.

Formalization: Let $\mathcal{A}_{\text{irr}}(s) = \{a : H(s'|s,a) > \epsilon, \forall \text{ sample sizes}\}$ be the set of actions with irreducible stochasticity from state s. For any strategy g that maximizes a signal $q(s,a)$ where q is monotone in model uncertainty: $\Pr[g(s) \in \mathcal{A}_{\text{irr}}(s)] \geq |\mathcal{A}_{\text{irr}}(s)|/|\mathcal{A}|$. For argmin: $\Pr[g_{\text{argmin}}(s) \in \mathcal{A}_{\text{irr}}(s)] = |\mathcal{A}_{\text{irr}}(s)|/|\mathcal{A}|$ (uniform in the limit). Targeted strategies have strictly higher probability of selecting irreducible transitions; argmin treats them equally.

Relationship to prior work: The noisy TV problem is known (Burda et al., 2019). Count-based robustness is known (Bellemare et al., 2016). Our contribution is empirical: (1) confirming the noisy TV problem applies to R1-compliant systems where death transitions are the "noisy TV," and (2) showing that 6 independent attempts to improve on argmin all fail for the same reason. The claim "argmin is locally optimal among memoryless count-based strategies in environments with irreducible stochasticity" is our empirical finding, not a published theorem.

Implication for R3: Self-observation (Theorem 2) requires the system to extract new structure from its own state after exploration saturates. But any mechanism that TARGETS specific state structures for observation will be vulnerable to the noisy TV problem — self-referential noise (patterns that look complex but carry no usable signal). The self-observation mechanism must be as blind as argmin is to transition entropy.

4.6 Constructive Characterization of the Feasible Region

Combining all formalized constraints, we characterize the class of F that satisfies R1-R6 + validated U-constraints simultaneously.

The minimal frozen frame F decomposes as:

$$F(s)(x) = \texttt{store}(s, \texttt{select}(s, \texttt{compare}(s, x)))$$

where: - $\texttt{compare}(s,x)$: produces similarity/distance scores between x and elements of s - $\texttt{select}(s, \text{scores})$: chooses an element of s (or triggers creation of a new element) - $\texttt{store}(s, \text{selection})$: updates s with the result

Constraints on each operation:

Operation	Must satisfy	Cannot do
compare	Lipschitz in x (U20). Varies with s (R3). Encodes $x - \mathbb{E}[x]$ (U16).	Rearrange topology (T3). Use external signal (R1).
select	Deterministic given s and scores. Hard selection, not soft blend (U8, provisional).	Converge to fixed policy (U22 + Thm 1).
store	Only add, never remove (U3). Irredundant (R6). Unbounded (U17).	Create redundant components. Delete existing components.

Phase structure (from Theorem 1):

The trajectory of s has two alternating phases: - **Phase A (amplification):** U7 drives toward the dominant mode. compare and select become increasingly predictable. The system exploits accumulated structure. - **Phase B (disruption):** U17 growth adds new components. Via R3, new components change compare and select. The dominant mode shifts. The system explores new structure.

The oscillation between phases is perpetual (Theorem 1). The PERIOD is a degree of freedom — fast oscillation produces chaotic behavior, slow oscillation produces near-convergent behavior with occasional disruptions.

After exploration saturates (Theorem 2):

Phase B must draw new components from s itself (self-observation), not from x. The store operation must be able to create components of TYPE different from the original observation-derived components — meta-components that represent patterns, compressions, or properties of the existing state.

This means S is not homogeneous. It contains at least two kinds of elements: 1. **Observation-derived** components (nodes from $\pi(x)$, edges from transitions) — bounded by the finite environment. 2. **State-derived** components (patterns extracted from the trajectory of type-1 components) — potentially unbounded, satisfying U17 after saturation.

What remains undetermined:

The decomposition into compare-select-store is the STRUCTURE of F. The specific IMPLEMENTATIONS of each operation are the degrees of freedom (Section 6). The feasible region is the set of all $(compare, select, store)$ triples that satisfy the constraints in the table above.

Non-emptiness: The feasible region for compare-select-store with observation-derived components is non-empty — the graph + k-means + argmin system occupies it (Section 5.1). The feasible region INCLUDING state-derived components (self-observation) is the open question. No system has been shown to satisfy R6 after exploration saturates.

Relationship to prior work: The compare-select-store decomposition resembles the read-match-write cycle of Neural Turing Machines (Graves et al., 2014) and the content-based addressing of memory-augmented neural networks. The key difference: in NTMs, the controller is trained by backpropagation (violates R1/R2). In our framework, the controller IS F — frozen, minimal, and non-trainable. The state s does all the adapting.

5. Experimental Evidence

5.1 Navigation (546+ experiments)

All 3 ARC-AGI-3 games solved at Level 1:

Game	Mechanism	Result	Steps
LS20 Level 1	LSH k=12 + centered_enc + avgpool16, 4 actions, argmin	10/10 at 200K; 9/10 at 120K	471 (easy seed) to 137K (hard seed)
FT09	k-means n=300, 69 actions (64 click grid + 5 simple), argmin	3/3 at 50K	157, 2035, 3840
VC33	k-means n=50, 3 actions (zone discovery), argmin	3/3 at 30K	18, 82, 144

Unifying mechanism: graph + edge-count argmin + correct action decomposition. The mapping (observation → node) and action space decomposition are the variables. The graph + argmin is the constant across all winning configurations.

Action space finding: The native API presents 6 actions (FT09) and 1 action (VC33). The effective action space is 69 and 3 respectively. The mismatch between API-visible actions and effective actions was the key unsolved problem from Steps 467-502. Zone discovery (Step 505) and grid-click expansion (Step 503) resolve it.

Apparent ceiling resolved: The 6/10 ceiling at 50K on LS20 (Steps 459-483) was a step budget artifact. Hard seeds need 35K-115K steps vs 5-20K for easy seeds. Step 484 confirmed 10/10 at 200K. The 6/10 figure should not be cited as a fundamental limit.

Encoding requirements: centered_enc ($x - \text{mean}(x)$) is load-bearing across 2 families (codebook: prevents cosine saturation; LSH: prevents hash concentration). avgpool16 (64×64 → 16×16 = 256D) is required for LS20 — raw 4096D with k=16 gives 1/5 vs 6/10 for avgpool16 (Step 466).

5.2 Level 2 Failure as Feasibility Violation

LS20 Level 2 is the only unsolved problem. The "259-cell ceiling" (Steps 486-492) was a TIME LIMIT, not a topological barrier. Steps 528-529 showed sublinear growth continues: 259 cells at 50K → 439 at 740K, growth rate ~2 cells/100K at 740K. At k=16, reachable set expands to 1094 cells at 200K (Step 531), 1267 with self-refinement (Step 542). Level 2 remains unreached at all tested budgets and granularities.

Two families confirm identical behavior: - LSH k=12: 439 cells at 740K, 0/N Level 2 (Steps 486-492, 528-529) - L2 k-means n=300: 286-cell plateau, 0/3 Level 2 (Step 493) - Recode k=16: 1267 cells at 500K, 0/5 Level 2 (Step 542)

All edge manipulations reduce coverage below the pure argmin baseline (argmin=259 > decay=241 > death-avoidance=227 > death-seeking=196, Steps 489-492). Six targeted exploration strategies

all perform worse than argmin (Section 4.5). The reward region is beyond the argmin-reachable frontier regardless of mapping architecture, partition granularity, or budget.

The state graph reveals why (Step 550). At 500K steps with Recode k=16: 942 live cells, fully connected (1 component), 13695 edges. The agent circulates in a 364-node **active set** (29% of nodes). 834 nodes (67%) were visited early and abandoned. 134 (node, action) pairs have low entropy ($H < 0.1$) and only 10 observations each — these are the **deterministic frontier**: edges the agent has barely tried, leading to potentially unexplored regions. 67% of all (node, action) pairs have $H > 1.0$ (noisy TV transitions). Argmin is locally optimal (picks the least-visited action from the current node) but globally trapped: from any node in the active set, all actions are well-explored (> 100 observations), so argmin cycles within the attractor. The abandoned nodes with deterministic frontier edges are unreachable because argmin never drives the agent toward them.

Implication: L2 is not a resolution problem (942 cells is more than enough). It is not a budget problem (growth rate is ~2 cells/100K at 740K). It is a **policy problem**: argmin creates an attractor basin in the active set, and the L2 path requires escaping this basin to reach the deterministic frontier in the abandoned set. This directly validates Theorem 2: self-observation is needed to detect the attractor and escape it. ℓ_π (partition refinement) refines within the attractor (Step 549) but cannot break out.

Relationship to Section 4: Edge counts grow (U17 formally satisfied) but marginal counts in the active set are redundant (R6 violated). The 134 deterministic frontier edges are the non-redundant growth targets — but argmin cannot reach them.

5.3 Architecture Family Summary

11 families tested across 546+ experiments.

Family	Experiments	Navigation result	Kill reason
Codebook / LVQ	~435	LS20 Level 1 won (Step 82). FT09 won (~Step 83, 69 click classes). VC33 won (Step 375, 3 zones). Level 2 never solved.	Reached capability ceiling. Codebook ban 2026-03-18 (Jun's directive). S-class constraints (cosine-specific). Note: Step 375 VC33 win used handcrafted timer encoding (row 0 pixel count) — game-specific, not general. General zone mechanism confirmed by Step 505.

Family	Experiments	Navigation result	Kill reason
Reservoir	~20	0/N. Steps 438-465.	Rank-1 collapse (U7). Temporal inconsistency = local continuity failure (U20). Cells don't map nearby observations to the same node.
Grid graph	8	0/N. Steps 446-451.	No local continuity: grid topology is arbitrary, not observation-derived. Direct U20 violation.
LSH	~30	LS20: 10/10 at 200K (Step 484). FT09/VC33: topology gap confirmed (Steps 495-498).	Not killed. Action space limitation confirmed (Steps 495-498). Requires 69-action expansion for FT09.
kd-tree	1	0/N. Step 452.	Node splits destroy edges (persistence violated, U3). Immediate kill.
Cellular automata	3	0/N. Steps 449-451.	Degenerate mapping: single attractor state, no cell diversity. Immediate kill.
LLM	1	0/1. Step 462.	Action collapse: 100% ACTION1, no exploration. Preliminary result (n=1).
L2 k-means	~9	LS20: 5/10 at 50K (Step 475), comparable to LSH. FT09: 3/3 (Step 503). VC33: 3/3 (Step 505).	Not killed. Active family. Zone discovery + argmin is general mechanism.
Bloom filter	1	0/10. Step 494.	No edge memory: observation→cell without edge state = near-random walk. Confirms graph mechanism is load-bearing.

Family	Experiments	Navigation result	Kill reason
SplitTree	5	LS20: 3/3 L1 (Step 537, combined fix: edge transfer + thresh=64). Chain: KILLED (Step 538, deterministic splitting fragments the graph across domains).	Self-partitioning binary tree from transition divergence. Navigates L1 but fully deterministic splitting creates fragile domain separation. 11th distinct family.
Recode	3	LS20: **5/5 L1** (Step 542) — best non-codebook result. Chain: 2/3 L1 with per-domain centering (Step 546).	Not killed. LSH k=16 + passive self-refinement from transition statistics. First substrate where the mapping self-modifies AND navigation succeeds reliably. See Section 5.4.5.

Cross-family findings: - Local continuity (U20) is the strongest kill criterion: 4 families fail cleanly on this axis (grid graph, CA, reservoir, kd-tree). - centered_enc is load-bearing in 2 families (codebook, LSH) for different failure modes — confirmed universal (U16). - Graph + edge-count argmin is the constant across all winning navigation configurations. No family has won without it. - **Noisy TV is universal:** Entropy-seeking (Absorb substrate, Steps 539-541) drives the agent into lethal states because death transitions have maximum irreducible entropy (Burda et al. 2019). 6 independent targeted exploration strategies tested, all worse than argmin (Section 4.5). The Absorb substrate (LSH + entropy-driven action selection + BFS routing) is the 12th tested architecture — KILLED for noisy TV. Not counted as a distinct family because it shares LSH's spatial engine; the failure is in action selection, not mapping. - Classification: **R1-compliant classification is unsolved.** Step 432 (codebook) achieves 94.48% and Step 444b (graph) achieves 93.34% — but both receive ground-truth labels on every training step (supervised NNC). Without external labels, accuracy drops to chance (~10%). LSH k=16 achieves NMI=0.599 on CIFAR-100 (Step 547) — the encoding has class signal, but no mechanism extracts it without external labels. The R1 constraint (no external objective) and the classification task are fundamentally incompatible with all tested architectures.

5.4 The Chain Benchmark

The experiments in Sections 5.1-5.3 test single benchmarks in isolation. The real test is the **chain**: a sequence of heterogeneous benchmarks with one continuous state and no reset.

```
CIFAR-100 → Atari → ARC-AGI-3 → CIFAR-100
```

Each benchmark tests a different capability that specialized systems win: - **CIFAR-100**: classification (DNNs/LLMs win). 100 classes, 32×32 RGB images. R1 prohibits external labels — the

substrate must classify by its own dynamics. - **Atari**: navigation/control (RL wins). Reward-driven sequential decision-making. R1 prohibits external reward — the substrate must navigate by exploration. - **ARC-AGI-3**: reasoning (humans win). Interactive games requiring perception, action, and adaptation. Tested in Sections 5.1-5.3.

What the chain tests that single benchmarks don't: 1. **U3 under task switching:** Does the growth-only state (nodes, edges) from CIFAR-100 survive ARC-AGI-3? Or does ARC-AGI-3's state contaminate CIFAR-100's representations? 2. **R4 across benchmarks:** After ARC-AGI-3 modifies the state, does CIFAR-100 performance degrade? R4 requires that modifications are tested against prior capability. 3. **U11 in sequence:** The substrate must classify (argmax-like) AND navigate (argmin-like) with one state, one mechanism. The chain forces both in sequence, not in parallel.

Protocol: 5-minute phases per benchmark. 1-pass CIFAR (10K images), 5-min per ARC game, 1-pass CIFAR return.

5.4.1 Negative Transfer and Dynamic Growth (Steps 506-508) Frozen centroids from CIFAR break ARC navigation (Step 506: 0/1 at 50K vs 5/10 baseline). This is negative transfer (Rosenstein et al. 2005): CIFAR-trained centroids impose a partition hostile to game frame navigation. ARC frames map to 99/300 CIFAR centroids — the wrong topology.

Dynamic centroid growth fixes this (Step 507: WIN@11170). When new centroids spawn for observations distant from existing ones (L2 > 0.3), CIFAR and ARC centroids naturally separate: CIFAR images have L2 mean=4.3, ARC frames L2 mean=0.5. Zero cross-task interference (CIFAR accuracy delta = -0.05pp).

The full chain passes (Step 508): LS20 WIN@11170, FT09 WIN@8075, VC33 WIN@11, CIFAR delta=-0.01pp. FT09 reuses LS20 centroids almost entirely (2 new centroids spawned). Domain separation is automatic.

5.4.2 The Threshold Tension (Steps 509-513) The encoding (avgpool16 + centered) contains class signal: NMI=0.42 at threshold=3.0 with 2701 centroids (Step 512). NMI climbs monotonically with centroid count (Step 510). But the spawn threshold is incompatible across domains: CIFAR needs threshold ≥ 3.0 for meaningful clustering; ARC needs threshold ≤ 0.5 for navigation. One fixed threshold cannot serve both.

Domain-adaptive threshold (Step 513) auto-calibrates for ARC (median=0.308, matching optimal fixed) but fails for CIFAR — local density in sparse 256D does not reflect class structure. The encoding, not the threshold mechanism, is the bottleneck for classification.

5.4.3 Cross-Family Chain Replication (Steps 515-525) **Negative transfer is universal** (Step 515): K-means with CIFAR-fitted centroids fails on LS20 (3/300 cells) — same collapse as codebook Step 506. Not mechanism-specific.

LSH chain corrected (Steps 516, 523): Single-seed Step 516 (WIN@1116) was an outlier. Multi-seed Step 523 reveals LS20 0/3 — CIFAR edges on actions 0-3 contaminate LS20's 4 directional actions. Action-scope isolation works only for expanded action spaces (FT09: 69 actions, 3/3; VC33: 3 zones, 3/3) but not for baseline LS20.

Frozen k-means collapses (Step 522): All FT09/VC33 frames map to 1/300 centroids. FT09 3/3 is round-robin exploitability, not transfer.

5.4.4 Algorithm Invariance (Steps 521, 524, 525) **Prior work:** Bisimulation relations (Givan et al., 2003) formalize when two MDP states are behaviorally equivalent — states with the same reward and transition structure over bisimilar successors. MDP homomorphisms (Ravindran & Barto, 2004) extend this to structure-preserving maps between MDPs. State abstraction theory (Abel et al., 2017) characterizes which abstractions preserve near-optimal policies. All concern equivalence of STATES. Our finding is about equivalence of REPRESENTATIONS of the same state — a simpler, more specific result.

Proposition 3 (Representation Invariance): Let $R : \mathcal{H} \to \mathbb{R}^{|A|}$ be a representation mapping transition history $\mathcal{H}$ to a per-action summary vector, and let $g(s) = \text{argmin}_a R(\mathcal{H})_a$. If R is *count-monotone* — i.e., $N(s,a) > N(s,a') \Rightarrow R(\mathcal{H})_a > R(\mathcal{H})_{a'}$ where $N(s,a)$ is the visit count — then g produces identical action sequences regardless of the specific form of R.

Proof: Argmin depends only on the ordering of $R(\mathcal{H})_a$ across actions. Count-monotonicity preserves this ordering. Any two count-monotone representations select the same action. □

Empirical confirmation. Four representations tested on LS20:

Representation	Step	Result	Why count-monotone
LSH graph (edge dict)	459	6/10	$R_a = \sum_n E(s,a,n)$ — direct count
Hebbian weights (matrix W)	524	5/5 (1 trajectory)	$R_a = (W^T x)_a$ — accumulated weight $\propto$ frequency when x is deterministic per state
Markov tensor (T[c,a,c'])	525	8/10	$R_a = \sum_j T[s,a,j]$ — transition total = count
N-gram (history buffer)	521	4/5 (N=20)	Frequency estimate from recent window — converges to count ratios

All converge to **argmin over visit frequency**. Score variations (6/10 vs 8/10) are due to hash randomness and budget, not algorithmic differences.

Relationship to prior work: The proposition itself is mathematically trivial — argmin over orderings is order-invariant. The contribution is empirical: confirming that four architecturally distinct representations ARE count-monotone in practice, and that no representation bias introduces a qualitatively different algorithm. This is NOT bisimulation (which equates states) nor MDP homomorphism (which equates MDPs). It is a sufficient-statistic result: the visit-count ordering is a sufficient statistic for argmin action selection, and all tested representations preserve it.

Implication: The search space for new action-selection mechanisms is constrained. Any count-monotone representation converges to argmin. New representations are unlikely to produce new algorithms unless they introduce a qualitatively different selection rule (not argmin over accumulated state). Combined with Section 4.5 (argmin is robust to noisy TV), this means: the action-selection problem is solved for navigation — argmin is both optimal among tested strategies and invariant across representations. The open problem is self-observation (Theorem 2), not action selection.

5.4.5 Self-Refinement and Navigation Reliability (Steps 534-546, Recode) Recode is LSH k=16 with passive self-refinement: when a cell produces inconsistent transition outcomes (high entropy in $G[(n,a)]$), the substrate learns a separating hyperplane from the mean difference between the two most frequent transition patterns. This splits the cell into two children, each

mapping to a distinct transition regime. Refinement is conservative: max 3 splits per 5000 steps, MIN_OBS=8 per transition pair.

Results:

Configuration	Result	Cells	Steps
LSH k=16 (no refinement)	3/3 L1 at 200K (Step 531)	1149	200K
Recode k=16 (with refinement)	**5/5 L1** at 500K (Step 542)	1267	500K
Recode chain (per-domain centering)	2/3 L1 (Step 546)	—	500K
Recode chain (global centering)	0/5 (Step 543)	—	—
LSH k=16 (no centering)	0/5 (Step 544)	62	—

Analysis: The 5/5 vs 3/3 improvement is confounded with budget (500K vs 200K). Self-refinement adds ~10% cells (1267 vs 1149) — modest. The dominant factor in reachable set expansion is k (partition granularity), not refinement. However, Recode is the first substrate where the observation→cell mapping self-modifies AND navigation succeeds reliably. This is partial R3 compliance: the mapping (a component of F) changes based on the state (transition statistics). It is NOT full R3 — the refinement algorithm itself is fixed.

Centering tension resolved (Steps 543-546): Centered encoding ($x - \bar{x}$) is required for navigation (U16: without it, 62 cells, 0/5). But global centering kills domain separation in the chain — CIFAR and LS20 frames hash to shared nodes (Step 543). **Per-domain centering** resets the running mean on domain switch (on_reset signal). Result: 2/3 L1 on the chain, with s0 navigating FASTER than clean (L1@12201 vs 29691). R1-compliant: on_reset is a game event, not an external domain label. The 2/3 vs 5/5 reliability gap on the chain remains unexplained.

Classification (Step 547): Recode's self-refinement is inert on classification — single-pass image presentation never triggers refinement (MIN_OBS=8 per node-action pair is never met with ~1.4 images per node). NMI=0.599 is pure k=16 LSH. Self-refinement is a navigation mechanism, not a general learning mechanism.

R6 diagnostic (Step 548): Does self-refinement satisfy R6 (irredundancy) after exploration saturates? Prediction: <10% of post-saturation refinements change argmin action. **Result: 89.5% — prediction inverted.** Mechanism: when a node splits, one child inherits a fresh edge table; argmin defaults to the least-visited action (typically action 0). Since the pre-split action is rarely action 0 in well-explored cells, the split forces fresh exploration from that partition. 357 total refinements over 599K steps, 304 changed action selection. R6 is satisfied in the narrow sense: each refinement is behaviorally irredundant. Whether this produces new REACHABLE states (not just re-exploration of the same frontier) is tested by Step 549 (trajectory divergence).

R6 and trajectory divergence (Step 549): Recode vs plain LSH (same hash, same seed): Jaccard overlap declines from 0.951 (50K) to 0.798 (300K). But the divergence is entirely partition-based, not frontier-based: at every checkpoint, the number of LSH-unique cells exactly equals the number of Recode splits ($|unique_{LSH}| = |sp_{Recode}|$). Each split retires a parent from Recode's live set; that parent remains in LSH's set. Recode-unique cells are children of splits — finer partitions of the same spatial region, not new areas of the game world. Recode ends with FEWER live cells (991 vs 1115): trading breadth for depth.

Combined interpretation (Steps 548-549): ℓ_π self-modification satisfies R6 behaviorally (89.5% of splits change argmin) but not frontierally (no new game states reached). The 5/5 vs 3/3 reliability improvement comes from finer state discrimination within the same explored frontier — the agent distinguishes states that were previously confused, enabling more reliable navigation. This is a genuine improvement but does not address Theorem 2's requirement for self-observation after saturation: partition refinement cannot discover states BEYOND the original hash partition's reachable set. ℓ_π is necessary but not sufficient; ℓ_F remains the open question.

Non-codebook experiment count: ~99 (vs ~435 codebook). Ongoing scale-up.

6. Degrees of Freedom

The formalization identifies what the constraints REQUIRE but also what they leave UNDETERMINED. These degrees of freedom define the experiment space for the next phase.

#	Degree of Freedom	What's constrained	What's free	Source
1	Growth function ϕ	Must be monotonic, unbounded (U17)	What grows — nodes, edges, meta-state, or something else	Sec 3.2
2	Oscillation period	System cannot converge (Thm 1)	How quickly growth disrupts the dominant mode	Sec 4.1
3	Meta-rule F	Must be non-constant (R3), time-invariant (U1), minimal (R6)	The specific form of compare-select-store	Sec 3.2
4	Growth-rule coupling	Growth changes the update rule (R3)	Local vs global: does a new node change nearby rules or all rules?	Sec 3.3
5	Metric d_X on observations	Must make π Lipschitz (U20); can refine but not rearrange (T3)	Which metric: L2, cosine, learned, adaptive	Sec 3.3
6	Partition resolution	Nodes must be well-separated (Prop 2)	How many nodes: determined by spawn threshold vs observation distribution	Sec 3.3

#	Degree of Freedom	What's constrained	What's free	Source
7	Growth mechanism	Structure only grows (U3), without bound (U17)	Spawn-on-novelty vs implicit edge accumulation vs other	Sec 3.3
8	Self-observation target	Must extract irredundant structure from s (Thm 2)	What to extract: temporal patterns, graph properties, meta-state	Sec 4.3
9	Self-observation $\rightarrow$ rule modification	New structure must change f (R3)	How: does detecting stuckness change exploration? Does cycle detection trigger avoidance?	Sec 4.3
10	Fixed-point structure	Conjectured to exist (Sec 4.4)	What invariant is preserved under self-observation	Sec 4.4
11	Exploration/exploitation resolution	$g_{state} \neq g_{class}$ (U11), no mode switch (U1)	State-dependent behavior: what state feature triggers the switch?	Sec 3.4
12	Self-modification level	Must be $\geq \ell_\pi$ for R3 compliance (Sec 3.2)	How deep: ℓ_π (Recode) vs ℓ_F (full self-reference). Which component of F adapts, and by what trigger?	Sec 3.2

The central experimental question (DoF 8-9, 12): What specific self-observation mechanism produces irredundant growth after exploration saturates, and at what self-modification level? Recode shows ℓ_π is achievable; whether ℓ_F is necessary or achievable is the open question.

7. Discussion

7.1 Pairwise Consistency Audit

All formalized constraints were checked for mutual consistency. Identified tensions and their resolutions:

Tension	Constraints	Status
T1: Stationarity vs self-modification	U7 + R3	Open — U7 needs reformulation as instantaneous, not asymptotic
T2: Weak vs strong R3	R3 interpretation	**Resolved** — self-modification hierarchy (Sec 3.2). ℓ_π is partial, ℓ_F is full.
T3: Continuity vs self-modification	U20 + R3	Constrained — metric can refine but not rearrange topology
T4: Never delete vs no redundancy	U3 + R6	**Resolved** — irredundant growth (Sec 4.2). Every new component covers unique territory.
T5: Infinite growth vs finite environment	U17 + finite X	**Resolved** — Theorem 2. Growth shifts to state-derived components.
T6: Navigation vs classification	U11 + U24 + U1	Open — state-dependent behavior (DoF 11). No implementation yet.
T7: Self-observation vs noisy TV	Theorem 2 + Sec 4.5	Open — self-observation must avoid irreducible-noise traps. Constrains mechanisms.
T8: Centering vs domain separation	U16 + chain benchmark	**Resolved** — per-domain centering (Step 546). R1-compliant via on_reset.

No undiscovered contradictions. All tensions are either resolved (T2, T4, T5, T8), constrained (T3), or identified as open questions (T1, T6, T7). The constraint system is internally consistent.

7.2 What is proven

1. The system has no fixed point (Theorem 1). Self-modification is necessary.
2. In finite environments, the system must process its own state (Theorem 2). Self-observation is required.
3. The current graph + argmin mechanism exits the feasible region after exploration saturates.
4. Argmin over visit counts is robust to noisy TV and representation-invariant (Section 4.5, Proposition 3). Action selection for navigation is solved.
5. Self-modification of the encoding (ℓ_π) is achievable and improves navigation reliability (Recode, Step 542).

7.3 What is conjectured

6. The minimal self-observing substrate is a fixed point of F (Section 4.5).
7. The oscillation between U7 (convergence) and U17 (growth) is a limit cycle, not chaos.

7.4 What is open

8. Whether the full feasible region (R1-R6 + all validated U-constraints) is non-empty.
9. What self-observation mechanism satisfies R6 (irredundancy) while avoiding noisy TV traps (T7).
10. Whether ℓ_F (full self-reference) is achievable without infinite regress — or whether ℓ_π suffices.
11. How to resolve U11 + U24 + U1 (incompatible tasks, no mode switch) in a single system (T6).
12. R1-compliant classification — no substrate has achieved above-chance accuracy without external labels.

7.5 Honest assessment

The feasible region for Level 1 navigation is occupied — graph + argmin + correct action decomposition satisfies R1, R2, U1, U3, U17, U20 and solves all three games. Recode extends this to partial R3 (ℓ_π). But no system satisfies R3 in the strong sense (ℓ_F), R4 (no self-testing), or R6 after exploration saturates. The full R1-R6 region remains unoccupied. Whether it contains a point is the open question this paper frames but does not answer.

References

- Abraham, W. C. & Robins, A. (2005). Memory retention — the synaptic stability versus plasticity dilemma. Trends in Neurosciences, 28(2), 73-78.
- Bellemare, M. et al. (2016). Unifying Count-Based Exploration and Intrinsic Motivation. NeurIPS.
- Burda, Y. et al. (2019). Large-Scale Study of Curiosity-Driven Learning. ICLR.
- Fritzke, B. (1995). A Growing Neural Gas Network Learns Topologies. NeurIPS.
- Givan, R., Dean, T. & Greig, M. (2003). Equivalence Notions and Model Minimization in Markov Decision Processes. Artificial Intelligence, 147(1-2), 163-223.
- Graves, A. et al. (2014). Neural Turing Machines. arXiv:1410.5401.
- Jin, C. et al. (2020). Reward-Free Exploration for Reinforcement Learning. ICML.
- Kirsch, L. & Schmidhuber, J. (2022). Self-Referential Meta Learning. ICML.
- Kohonen, T. (1988). Self-Organization and Associative Memory. Springer.
- Maturana, H. & Varela, F. (1972). Autopoiesis and Cognition: The Realization of the Living.
- McCloskey, M. & Cohen, N. J. (1989). Catastrophic interference in connectionist networks. Psychology of Learning and Motivation, 24, 109-165.
- Pathak, D. et al. (2017). Curiosity-driven Exploration by Self-Supervised Prediction. ICML.
- Ravindran, B. & Barto, A. G. (2004). Approximate Homomorphisms: A Framework for Non-Exact Minimization in Markov Decision Processes. ICML Workshop.
- Rosenstein, M. et al. (2005). To Transfer or Not To Transfer. NIPS Workshop on Inductive Transfer.

- Rudakov, E., Shock, J. & Cowley, B. U. (2025). Graph-Based Exploration for ARC-AGI-3 Interactive Reasoning Tasks. arXiv:2512.24156.
- Schmidhuber, J. (2003). Gödel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements. arXiv:cs/0309048.
- van de Ven, G. M. & Tolias, A. S. (2024). Continual Learning and Catastrophic Forgetting. arXiv:2403.05175.
- Wang, Z. et al. (2019). Characterizing and Avoiding Negative Transfer. CVPR.